Rapport de Projet IA - M1 TI
Classification de Séries Temporelles

Lien vers Google Colab

En cas de probleme lien vers Google Colab de Dev

Lien vers le Github si besoin

LACATON Damien - BOUZOM Lukas

20 mars 2026

Table des matières

1 Introduction **2**
1.1 Objectifs du projet 2

2 Présentation et préparation des données **3**
2.1 Origine des données 3
2.2 Exploration visuelle 4
2.3 Préparation des Données 5
2.4 Analyse de la répartition des données 6

3 Modèle 1 : LSTM (Long Short-Term Memory) **7**
3.1 Fonctionnement 7
3.2 Architecture 7
3.3 Résultats et analyse 8
3.3.1 Performance par catégorie 9
3.3.2 Expérimentations et Optimisation 10

4 Modèle 2 : CNN 1D (Convolutional Neural Network) **11**
4.1 Fonctionnement 11
4.2 Architecture 11
4.3 Résultats et analyse 12
4.3.1 Performance par catégorie 13
4.3.2 Expérimentations et Optimisation 14

5 Modèle 3 : Time Series Transformer **15**
5.1 Fonctionnement 15
5.2 Architecture 15
5.3 Résultats et analyse 16
5.3.1 Performance par catégorie 17
5.3.2 Expérimentations et Optimisation 18

6 Expérimentation : Une variable supplémentaire **19**
6.1 Ajout de la variable "Energy Usage" 19
6.2 Résultats et Comparaisons 19
6.3 Analyse Résultat 21

7 Perspectives **22**

Chapitre 1

Introduction

Ce projet fait partie de l'UE IA pour les systèmes cyber-physiques en Master 1. L'objectif est de faire de la classification de séries temporelles. Nous avons à notre disposition des données représentent des traces d'activités de processeur (CPU), de disque et de mémoire vive (RAM).

Le but est de pouvoir identifier automatiquement de quel type d'activité il s'agit à partir de traces d'exécution.

1.1 Objectifs du projet

Le travail consiste à tester et comparer trois approches différentes :

— Les réseaux de neurones récurrents de type LSTM.

— Les réseaux de neurones convolutionnels (CNN 1D).

— Les modèles plus récents basés sur le mécanisme d'attention (Transformers).

Puis proposer des pistes susceptibles de répondre au problème identifié afin de déterminer si les séquences de traces d'exécution proviennent d'un navigateur web, d'un compilateur, d'un logiciel de traitement de texte, etc.

Chapitre 2

Présentation et préparation des données

2.1 Origine des données

Les données utilisées proviennent d'un dépôt GitHub public regroupant des traces d'exécution de CPU. Chaque échantillon correspond à un fichier CSV qui regroupe plusieurs colonnes de mesures du système, relevées à des intervalles réguliers.
`https://github.com/phuongntmse/ts-data-for-workload-classification`

CPU Workloads (Example)

Folder	Benchmark Tool	Execution Command
cpu-1	sysbench	`sysbench cpu --cpu-max-prime=99999999 --num-threads=4 run`
cpu-9	stress-ng	`stress-ng --cpu 4 --metrics-brief -v --cpu-method queens --timestamp -t 5m`
cpu-16	phoronix	`phoronix-test-suite batch-run benchmark compress-7zip`

Disk Workloads (Example)

Folder	Benchmark Tool	Execution Command
disk-2	fio	`fio --name=seqwrite --rw=write --bs=1K --size=1G ...`
disk-9	phoronix	`phoronix-test-suite batch-run benchmark compilebench`

Memory Workloads (Example)

Folder	Benchmark Tool	Execution Command
memory-2	sysbench	`sysbench memory --threads=4 --memory-block-size=1K ... run`
memory-5	phoronix	`phoronix-test-suite batch-run benchmark tinymembench`

2.2 Exploration visuelle

Avant de traiter les données, nous avons visualisé les signatures de chaque catégorie. Par exemple, une activité intense du CPU correspond à une charge élevée et continue du processeur, tandis qu'une activité du disque montre des pics lors des phases de lecture ou d'écriture. Enfin, l'activité de la mémoire se manifeste par une occupation assez constante de la RAM, sans les variations brusques observées pour le disque.

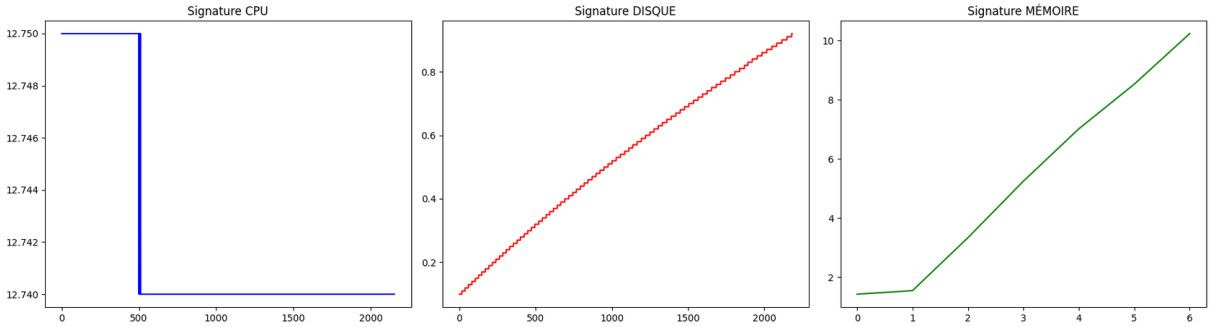

FIGURE 2.1 – Comparaison des signatures CPU, Disque et Mémoire

2.3 Préparation des Données

Pour que les modèles puissent apprendre efficacement, il faut appliquer plusieurs étapes aux données :

— **Sélection des données** : Nous retenons les 4 variables les plus intéressantes :

1. `CpuALL_usage` : Charge globale du processeur.
2. `memory_usage_perce` : Occupation de la RAM.
3. `read_rate_during_time` : Vitesse de lecture disque.
4. `write_rate_during_time` : Vitesse d'écriture disque.

— **Normalisation** : Afin de pouvoir comparer les valeurs plus facilement, nous les ramenons sur un intervalle commun entre 0 et 1 grâce à *MinMaxScaler*. Cela permet l'unicité des données, en évitant que certaines valeurs soient exprimées en pourcentage comme l'activité CPU et d'autres en octets comme la mémoire ; on pourra donc se les représenter dans un même graphe afin de permettre un meilleur apprentissage des modèles.

— **Fenêtrage** : Les données sont découpées en séquences de 50 unités de temps. Pour les modèles LSTM et CNN, c'est crucial afin qu'ils capturent l'activité sur un intervalle de temps plutôt qu'à un instant T. C'est ce bloc de 50 points qui sera donné en entrée aux modèles.

— **Labels** : Pour pouvoir entraîner nos modèles, il faut étiqueter nos données ; pour cela, nous avons déterminé que **CPU = 0, DISQUE = 1 et MÉMOIRE = 2**.

L'image ci-dessous représente les données après leur préparation. On observe que les quatre variables sont désormais à la même échelle, ce qui permet de les comparer sur une fenêtre de 50 points.

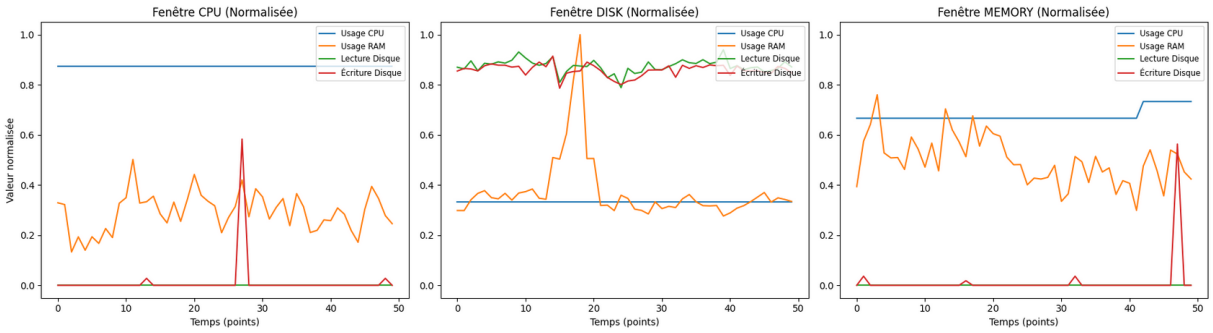

FIGURE 2.2 – Visualisation de fenêtres de 50 points après normalisation

La préparation des données nous a permis de constituer un dataset comportant :

— 35 928 séquences
— 50 unités de longueur
— 4 variables

2.4 Analyse de la répartition des données

Après avoir extrait toutes les séquences, nous avons analysé la répartition des données entre les trois catégories. On observe que les classes ne sont pas équilibrées : le CPU représente la majorité des données, alors nous disposons de moins de données pour la mémoire et le disque

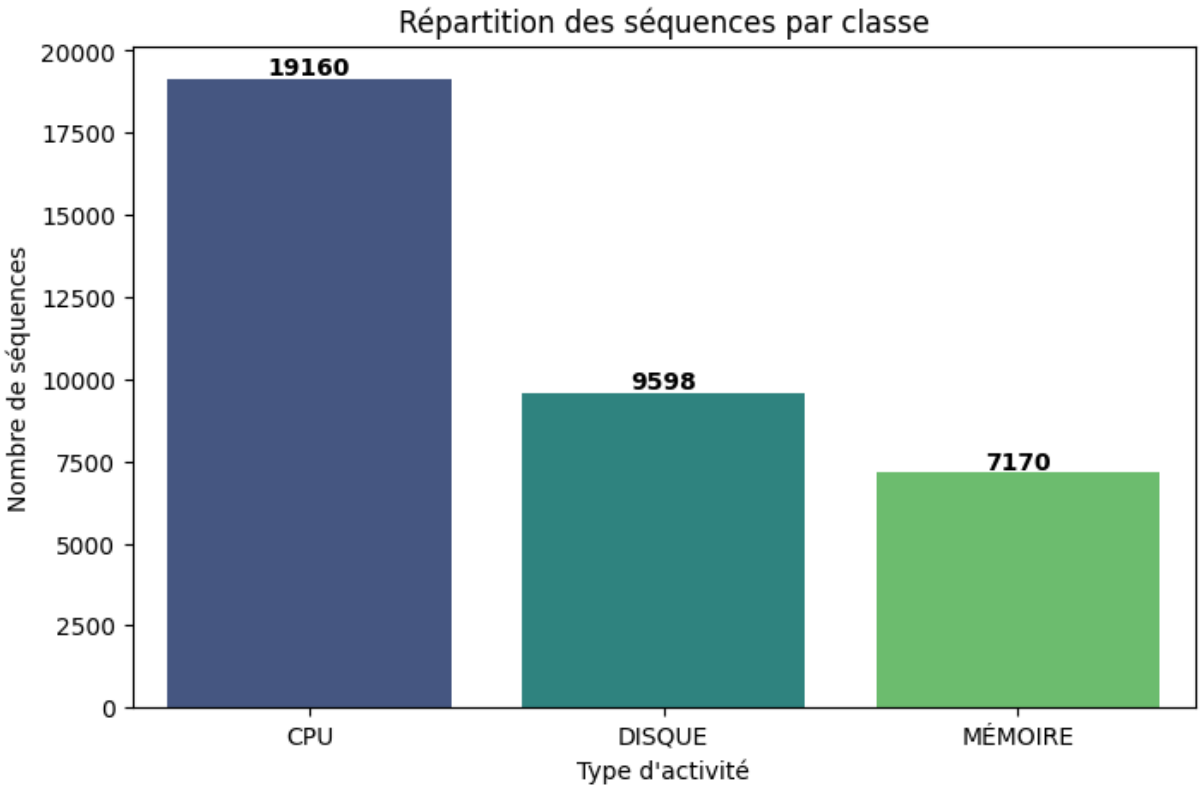

FIGURE 2.3 – Répartition des séquences par classe (CPU, Disque, Mémoire)

Ce déséquilibre entre les données aura une influence sur l'apprentissage. Les modèles auront tendance à mieux reconnaître la classe CPU, car ils ont plus d'exemples pour apprendre sa signature.

Chapitre 3

Modèle 1 : LSTM (Long Short-Term Memory)

3.1 Fonctionnement

Le LSTM est un réseau de neurones utilisé pour traiter des données séquentielles. Ce type de réseau a la particularité de pouvoir retenir des informations sur le long terme. Cela va être très utile pour reconnaître un pic d'activité CPU, car il faut analyser dans un contexte temporel, ici sur un intervalle de 50 unités de temps.

Son fonctionnement consiste à décider quelles informations passées doivent être conservées, mises à jour ou oubliées, ce qui nous permettra de reconnaître aisément une trace qui s'étale dans le temps (ex : une lecture disque).

3.2 Architecture

Nous avons construit un modèle avec les caractéristiques suivantes :

— **Entrée** : Des tenseurs sous la forme (50, 4).
— **Couche LSTM** : 64 unités pour capturer la dynamique des 4 variables.
— **Dropout (20%)** : Pour éviter le sur-apprentissage, 20% des neurones sont désactivés aléatoirement durant l'entraînement.
— **Couche Dense** : 32 neurones ReLU pour la prise de décision.
— **Sortie** : 3 neurones Softmax pour distribuer une probabilité pour chacune des 3 classes (CPU, Disque, Mémoire).

L'entraînement est réalisé sur 15 epochs et batch size de 64.

Avec les paramètres suivants :

— **Optimiseur** : Adam
— **Fonction de perte** : `sparse_categorical_crossentropy`

3.3 Résultats et analyse

Pour notre dataset, nous avons une répartition des données :

— **Entraînement** : `(28 742, 50, 4)`

— **Évaluation** : `(7 186, 50, 4)`

Les performances globales obtenues à la fin de l'exécution du modèle :

	Valeur
Précision finale (*Accuracy*)	**90,7%**
Précision de validation (*Val_ Accuracy*)	**91,2%**

La précision de validation est un peu supérieure à la précision d'entraînement ; cela indique que le modèle n'est pas en sur-apprentissage (overfitting).

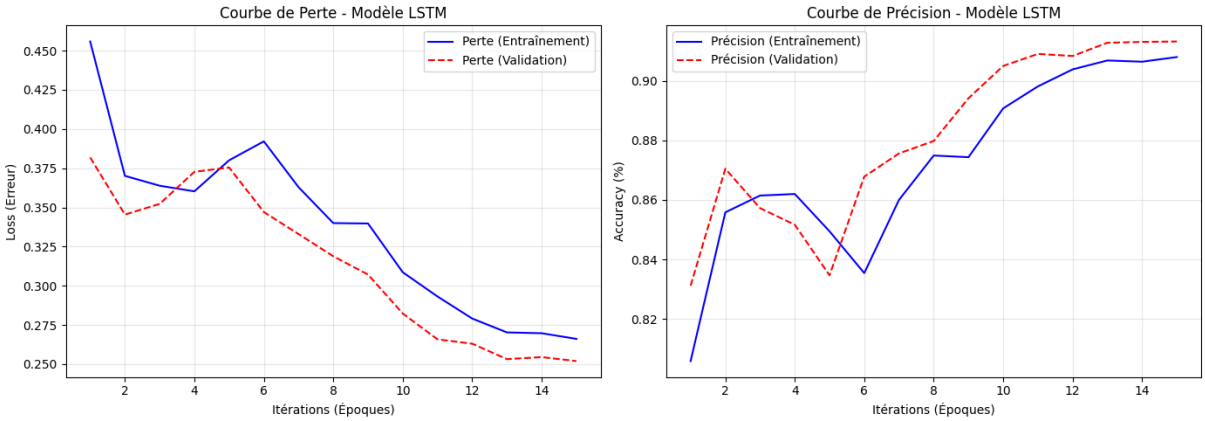

FIGURE 3.1 – Courbe d'apprentissage du modèle LSTM (Accuracy et Loss)

3.3.1 Performance par catégorie

— **Disque** : Le F1-score est de 1.00. La signature des accès disque est très simple à repérer pour ce modèle, qui se trompe rarement.
— **CPU** : Un F1-score de 0.92. Le modèle n'a donc aucun problème à reconnaître les activités CPU.
— **Mémoire** : C'est la catégorie où le modèle a le plus de mal, avec un rappel (recall) de **0.59**. Une partie des activités mémoire est confondue avec le CPU.

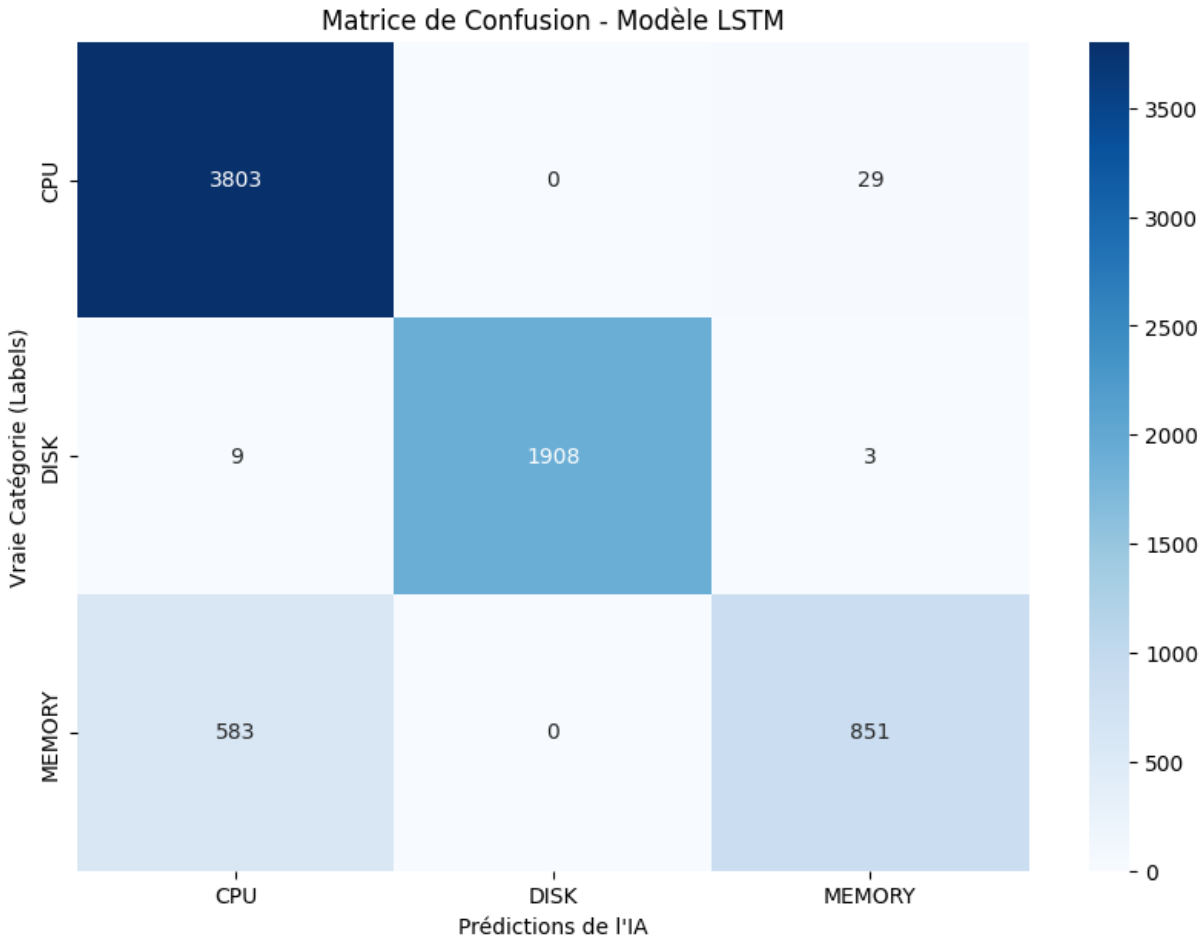

FIGURE 3.2 – Matrice de confusion du modèle LSTM

Le tableau suivant détaille la précision, le rappel et le F1-score pour chaque classe :

Classe	**Précision**	**Rappel**	**F1-score**	**Support**
CPU	0,86	0,97	0,91	3 832
DISQUE	1,00	0,99	1,00	1 920
MÉMOIRE	0,88	0,59	0,70	1 434
Exactitude (Accuracy)	**0,90**			7 186
Macro moyenne	0,91	0,85	0,87	7 186
Moyenne pondérée	0,90	0,90	0,89	7 186

3.3.2 Expérimentations et Optimisation

Dans le but d'améliorer les performances du modèle LSTM, nous avons effectué plusieurs expérimentations.

- **Taille de la fenêtre** : Est-ce que la taille de la fenêtre a une influence ?
- Test : 25 points ou 100 points.
- Observation : À 100 points, le modèle ne reconnaît pas la classe mémoire ; à 25 points, il y a du sur-apprentissage (overfitting).
- **Taille des lots (Batch size)** : Est-ce que la taille des lots a une influence ?
- Test : 32 ou 128.
- Observation : À 32, le modèle confond davantage la mémoire et le CPU ; à 128, il y a plus d'erreurs et aucune amélioration.
- **Dropout** : À partir de quel seuil observe-t-on du sur-apprentissage ?
- Test : 10% ou 30%.
- Observation : À 10%, pas de difference ; à 30%, on note un manque de données lors de l'apprentissage.

Ces tests confirment que nos valeurs initiales offrent le meilleur compromis entre rapidité d'exécution et précision.

Chapitre 4

Modèle 2 : CNN 1D (Convolutional Neural Network)

4.1 Fonctionnement

Le réseau de neurones convolutionnel (CNN) peut être utilisé sur des séries temporelles avec des convolutions 1D. Au lieu de regarder une série dans son intégralité, le modèle utilise des filtres pour détecter des motifs (pics, transitions brusques, etc.) sur la série temporelle.

L'avantage du CNN par rapport au LSTM est sa capacité à repérer des signatures de formes précises sans avoir besoin de les mémoriser.

4.2 Architecture

Pour le modèle CNN 1D, nous avons défini l'architecture suivante :

— **1re Couche Conv1D** : 64 filtres, une taille de noyau (kernel) de 3 ; pour repérer les motifs simples.

— **2e Couche Conv1D** : 128 filtres, une taille de noyau (kernel) de 3 ; pour repérer les combinaisons des motifs repérés par la couche précédente.

— **MaxPooling1D** : Réduit la dimension temporelle par 2, pour ne garder que les valeurs les plus importantes.

— **Dropout (20%)** : Pour éviter le sur-apprentissage, 20% des neurones sont désactivés aléatoirement durant l'entraînement.

— **Couche Dense** : 32 neurones ReLU pour la prise de décision.

— **Sortie** : 3 neurones Softmax pour distribuer une probabilité pour chacune des 3 classes (CPU, Disque, Mémoire).

L'entraînement est réalisé sur 15 epochs et batch size de 64.

Avec les paramètres suivants :

— **Optimiseur** : Adam

— **Fonction de perte** : `sparse_categorical_crossentropy`

4.3 Résultats et analyse

Pour notre dataset, nous avons une répartition des données :

— **Entraînement** : `(28 742, 50, 4)`

— **Évaluation** : `(7 186, 50, 4)`

Les performances globales obtenues à la fin de l'exécution du modèle :

	Valeur
Précision finale (*Accuracy*)	**91,6%**
Précision de validation (*Val_ Accuracy*)	**91,8%**

La précision de validation est un peu supérieure à la précision d'entraînement ; cela indique que le modèle n'est pas en sur-apprentissage (overfitting).

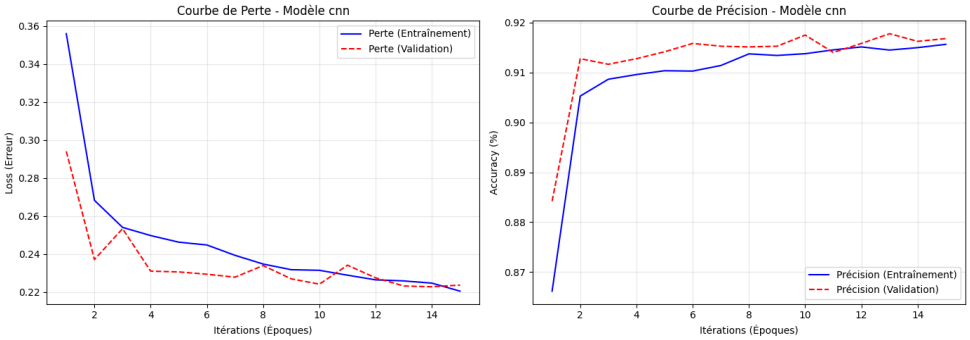

FIGURE 4.1 – Courbe d'apprentissage du modèle CNN 1D

4.3.1 Performance par catégorie

— **Disque** : Le F1-score est de 1.00. La signature des accès disque est très simple à repérer pour ce modèle.
— **CPU** : Un F1-score de 0.92. Le modèle n'a donc aucun problème à reconnaître les activités CPU.
— **Mémoire** : Ce modèle a egalment plus de mal a reperer les activiter mémoire avec un rappel (recall) de **0.60**. Une partie des activités mémoire est confondue avec le CPU.

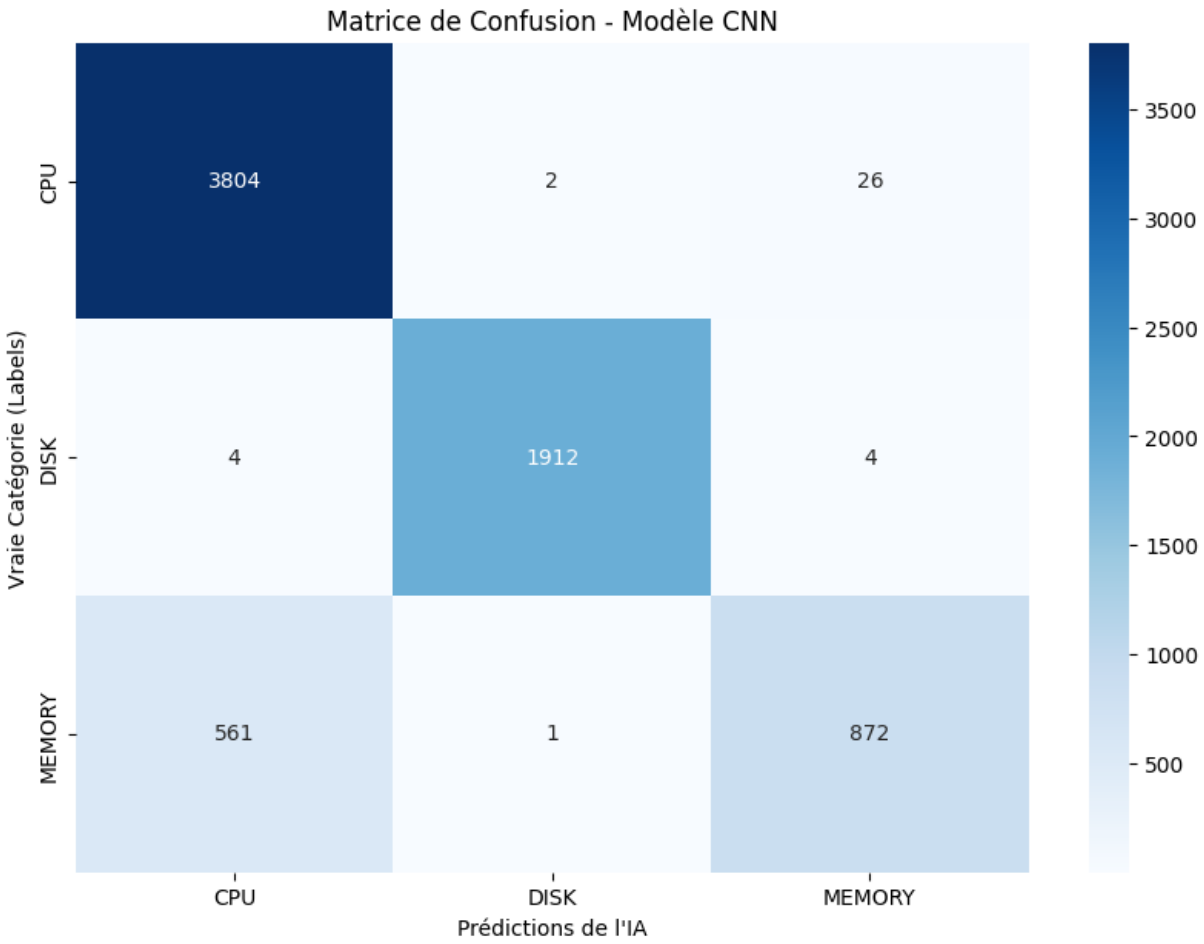

FIGURE 4.2 – Matrice de confusion du modèle LSTM

Le tableau suivant détaille la précision, le rappel et le F1-score pour chaque classe :

Classe	**Précision**	**Rappel**	**F1-score**	**Support**
CPU	0,87	1,00	0,93	3 832
DISQUE	1,00	0,99	0,99	1 920
MÉMOIRE	0,99	0,60	0,75	1 434
Exactitude (Accuracy)	**0,92**			7 186
Macro moyenne	0,95	0,86	0,89	7 186
Moyenne pondérée	0,93	0,92	0,91	7 186

4.3.2 Expérimentations et Optimisation

Dans le but d'améliorer les performances du modèle CNN, nous avons effectué plusieurs expérimentations.

- **Taille de la fenêtre** : Est-ce que la taille de la fenêtre a une influence ? Faut-il plus de contexte ?
- Test : 25 points ou 100 points.
- Observation : Aucune influence
- **Dropout** : À partir de quel seuil observe-t-on du sur-apprentissage ?
- Test : 10% ou 30%.
- Observation : À 10%, aucune différence ; à 30%, le rappel augmente de 0,01 à 0,02 point.
- **Taille des noyaux** : Un noyau plus grand permet de repérer des motifs plus longs.
- Test : 5 ou 9.
- Observation : L'ajout d'une couche `BatchNormalization()` stabilise l'apprentissage. Avec une taille de 5, aucun changement n'est observé ; avec une taille de 9, le modèle commet plus d'erreurs lors de l'évaluation, ce qui indique un sur-apprentissage (overfitting).
- **3e bloc** : Un bloc supplémentaire permet-il d'obtenir de meilleurs résultats ?
- Test : 256 filtres.
- Observation : Aucune amélioration de la précision n'est constatée, les confusions persistent. Cependant, une meilleure stabilité est obtenue.
- **Taux d'apprentissage (Learning Rate)** : Configuration d'un taux d'apprentissage plus haut ou plus bas que la valeur par défaut.
- Test : 0,005 ou 0,0005.
- Observation : À 0,0005, les erreurs augmentent lors de l'évaluation ; à 0,005, aucun changement significatif n'est relevé.
- **GAP** :
- Test : Remplacement de la couche `Flatten()` par `GlobalAveragePooling1D()`.
- Observation : Pas de changement significatif.
- **Poids des erreurs** : Rendre les erreurs sur la classe mémoire plus punitives pour augmenter l'attention du modèle sur celle-ci.
- Test : `poids_des_classes = {0: 1.0, 1: 1.0, 2: 2.0}`.
- Observation : La précision générale diminue, mais elle augmente pour la classe mémoire. Le modèle prédit la classe mémoire plus souvent, mais perd en précision sur la classe CPU, qu'il confond désormais avec la mémoire.

Ces tests ont permis de définir la meilleure configuration, privilégiant la stabilité et la régularisation : un modèle à 2 ou 3 blocs avec `BatchNormalization`, un *Dropout* de 0,3 et un noyau de taille 3 ou 5.

L'amélioration porte sur la classe **Mémoire** ; en rendant les erreurs plus punitives, le rappel passe à **0.69**, mais la précision générale diminue. La confusion avec le CPU est toujours présente. Les filtres de convolution semblent mieux repérer certaines signatures liées aux accès RAM que le mécanisme de mémoire du LSTM.

Chapitre 5

Modèle 3 : Time Series Transformer

5.1 Fonctionnement

Le Transformer est l'architecture qui utilise un mécanisme d'**Attention**. Le modele analyse simultanément tous les points de la fenetre et calcule un score d'importance par rapport à tous les autres points de la séquence

5.2 Architecture

L'architecture implémentée repose sur un bloc de codage, puis d'une tête de classification :

- **Multi-Head Attention** : 4 têtes d'attention. Chaque tête apprend reperer un types de motifs différents.
- **Layer Normalization** : Stabilise l'apprentissage
- **Feed-Forward Network (Conv1D)** : Deux couches de convolution avec un noyau de taille 1 qui fonctionne comme un réseau de neurones, pour les caractéristiques plus complexes.
- **Connexions Résiduelles** : Evite la dégradation des gradients et facilite l'entraînement du modèles.
- **Global Average Pooling 1D** : Compresse la séquence en un vecteur unique., qui sera utilisé pour la décision finale.
- **Couches Denses** : Une série de couches de neurone termine le traitement avant le **Softmax** à 3 classes.

5.3 Résultats et analyse

Pour notre dataset, nous avons une répartition des données comme les modèles précédents. Les resulats obtenues pour le Transformer sont :

Métrique	**Valeur**
Précision finale (*Accuracy*)	**89,1%**
Précision de validation (*Val_ Accuracy*)	**89,5%**

Absence de sur-apprentissage confirméegrace a la proximité des scores d'entraînement et de validation. Le modèle est légèrement moins performant que le CNN 1D.

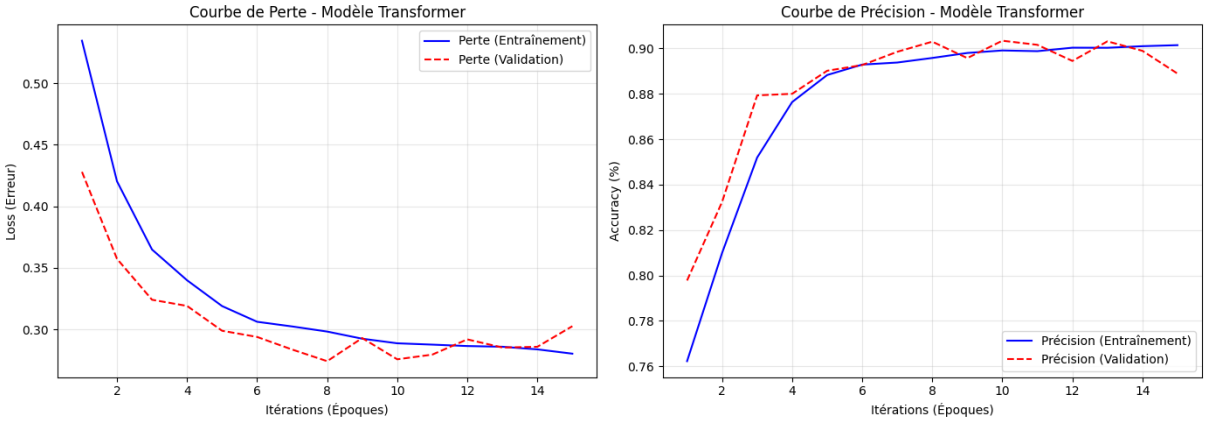

FIGURE 5.1 – Courbe d'apprentissage du modèle Transformer

5.3.1 Performance par catégorie

L'analyse détaillée montre que le Transformer partage les mêmes difficultés que le LSTM :

— **Disque** : F1-score de 1.00. Les signatures sont très bien identifiées.
— **CPU** : F1-score de 0.90.
— **Mémoire** : Comme pour les autres modèles, où il y a le plus d'erreur avec un F1-score de 0.71.

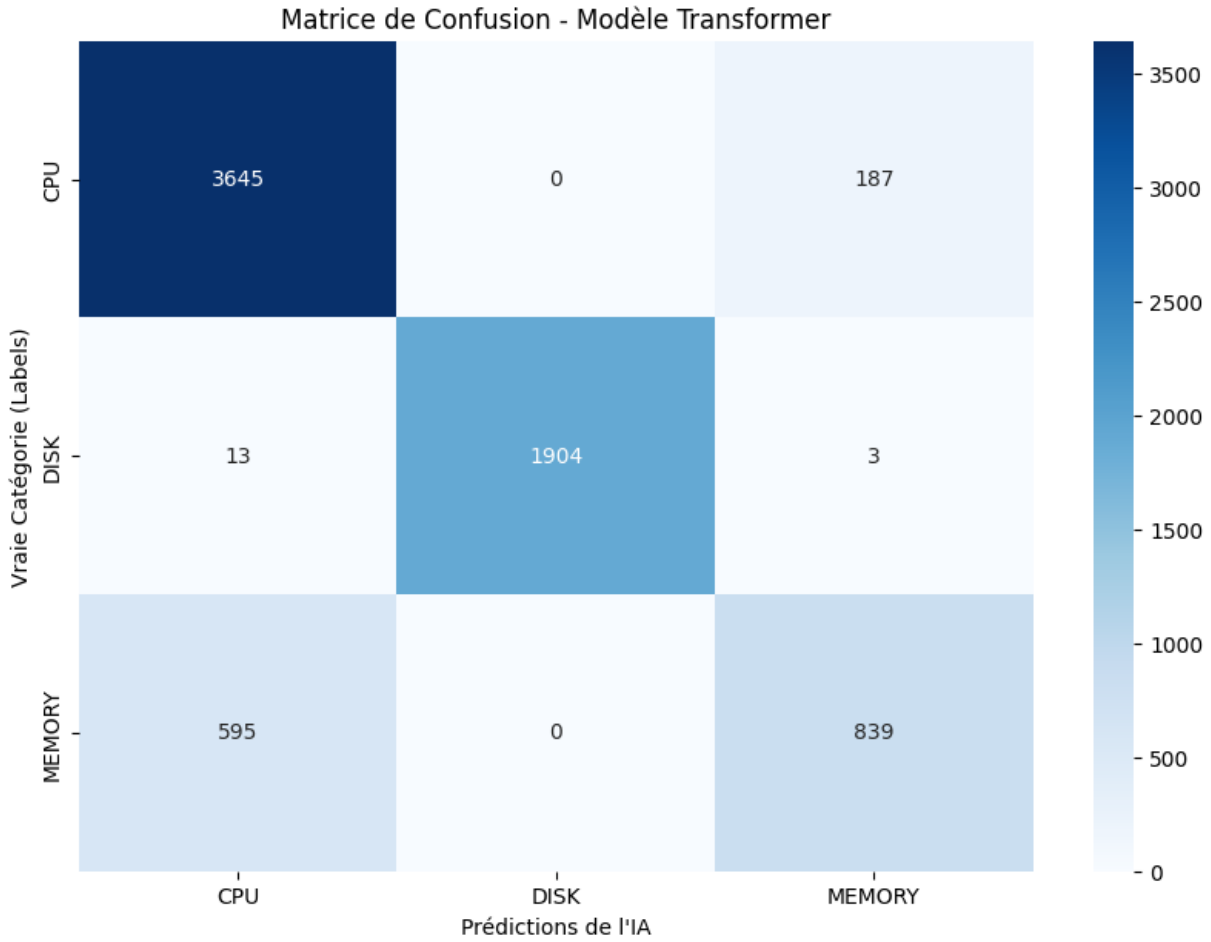

FIGURE 5.2 – Matrice de confusion du modèle Transformer

Le tableau suivant détaille la précision, le rappel et le F1-score pour chaque classe :

Classe	**Précision**	**Rappel**	**F1-score**	**Support**
CPU	0,86	0,98	0,92	3 832
DISQUE	1,00	0,99	0,99	1 920
MÉMOIRE	0,92	0,58	0,71	1 434
Exactitude (Accuracy)	**0,90**			7 186
Macro moyenne	0,93	0,85	0,87	7 186
Moyenne pondérée	0,91	0,90	0,90	7 186

5.3.2 Expérimentations et Optimisation

Plusieurs variantes ont été testées pour optimiser ce modèle :

— **Empilement (Stacking)** : Créer plus de blocs `transformer_encoder`.
— Test : Ajout de 1 ou 2 blocs.
— Observation : Pas d'amélioration significative.
— **Dimension** : Ajuster `head_size` et `ff_dim`.
— Test : Passage à 128.
— Observation : En passant à 128, le modèle n'a pas gagné en précision mais est devenu un peu plus stable.
— **Nombre de têtes** : Est-ce que le nombre de têtes a une influence ?
— Test : 8 têtes.
— Observation : Augmenter le nombre de têtes n'a pas amélioré la précision ; le modèle a commencé à présenter du sur-apprentissage (overfitting).
— **GAP** :
— Test : Remplacer ou ajouter une couche `GlobalMaxPooling1D`.
— Observation : En remplaçant le GAP par du `GlobalMaxPooling1D`, le modèle a perdu en précision lors de l'évaluation.
— **Taux d'apprentissage (Learning Rate)** :
— Test : 0,005 ou 0,0005.
— Observation : Pas d'amélioration significative.
— **Poids des erreurs** : Rendre les erreurs sur la classe mémoire plus punitives pour augmenter l'attention du modèle sur celle-ci.
— Test : `poids_des_classes = {0: 1.0, 1: 1.0, 2: 2.0}`.
— Observation : Résultats identiques à ceux du modèle CNN.

Tous ces tests montrent que l'architecture Transformer a atteint son plus haut taux de performance avec le jeu de données actuel. L'augmentation de la complexité du modèle n'a pas apporté d'amélioration significative de la précision et a même montré du sur-apprentissage (*overfitting*).

L'utilisation du *Global Average Pooling* était un meilleur choix que le *Global Max Pooling*, ce qui confirme que la moyenne globale des caractéristiques temporelles est plus efficace pour identifier les signatures.

Chapitre 6

Expérimentation : Une variable supplémentaire

6.1 Ajout de la variable "Energy Usage"

Nous avons utilisé quatre variables (CPU, RAM, Lectures et Écritures Disque).Étant donné que tous les modèles présentaient des difficultés à distinguer le CPU de la mémoire, l'hypothèse d'un manque de données a été formulée. Afin d'améliorer la distinction entre les classes, une cinquième variable a été ajoutée : `Energy_usage_during_time`. La consommation énergétique est supposée suivre des signatures spécifiques selon le composant sollicité.

6.2 Résultats et Comparaisons

Les trois modèles (LSTM, CNN 1D et Transformer) ont été réentraînés avec cette donnée supplémentaire. Les résultats indiquent une amélioration de la précision pour l'ensemble des modèles.

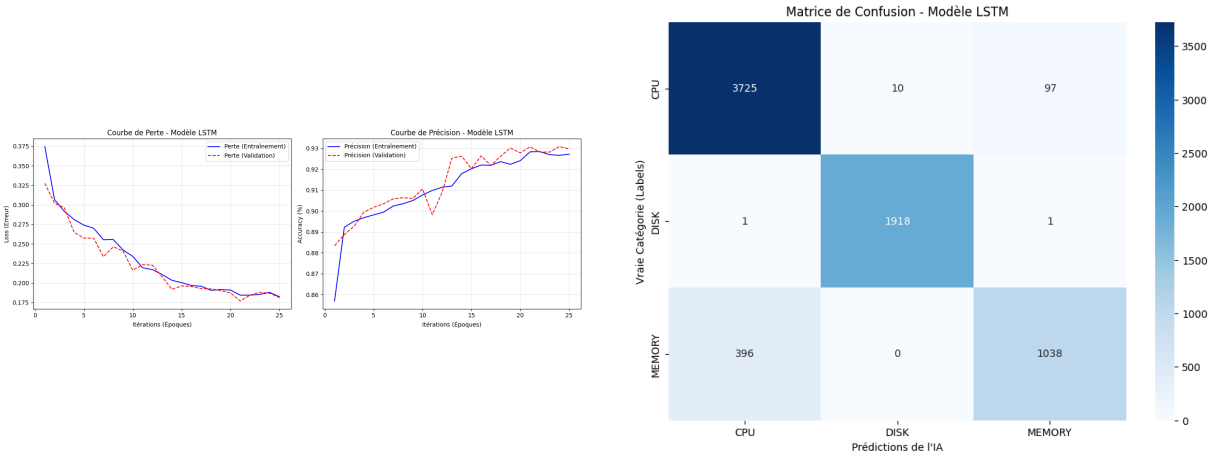

FIGURE 6.1 – Résultats LSTM - Avec variable Énergie

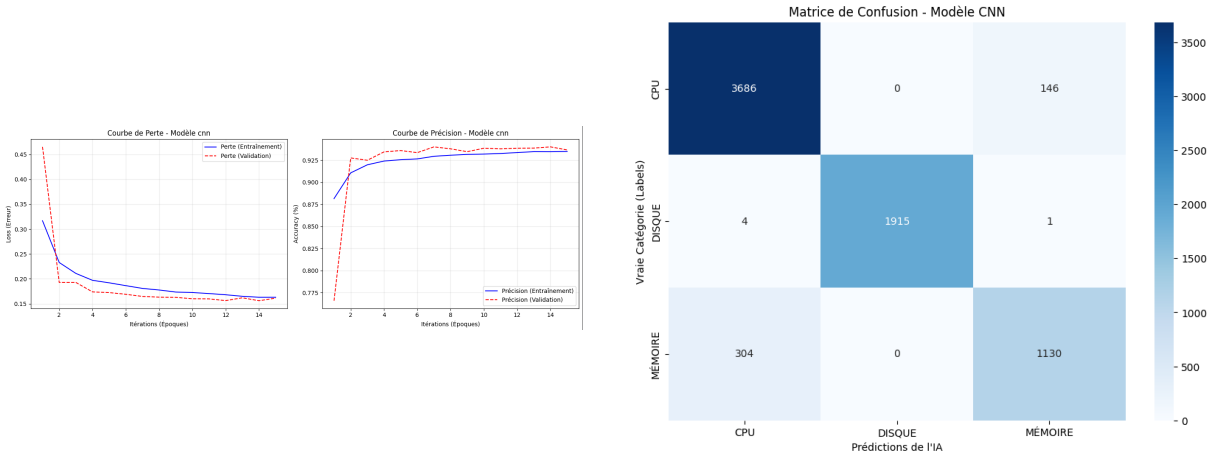

FIGURE 6.2 – Résultats CNN 1D - Avec variable Énergie

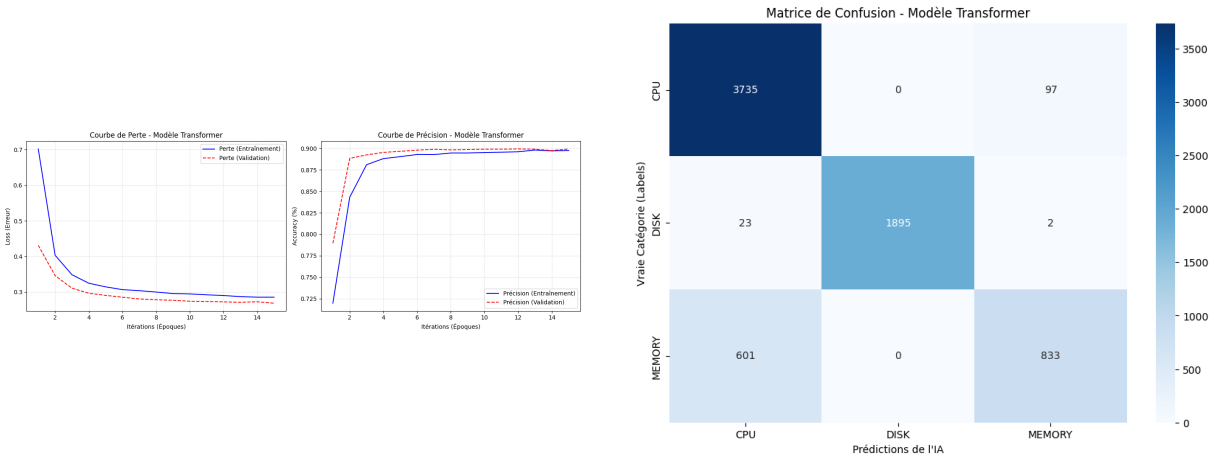

FIGURE 6.3 – Résultats Transformer - Avec variable Énergie

6.3 Analyse Résultat

Modèle	Accuracy	F1-Score CPU	F1-Score DISK	F1-Score MEM
LSTM	93%	0,94	1,00	0,81
CNN 1D	**94%**	0,94	1,00	**0,83**
Transformer	**94%**	0,94	1,00	**0,83**

TABLE 6.1 – Comparaison des performances des modèles avec 5 variables.

L'analyse des rapports de classification a permis d'éclairer certains points suite à l'ajout de la variable `Energy_usage_during_time` :

— **Séparabilité parfaite du disque** : Les trois modèles atteignent désormais un score de 1,00 (précision et rappel) sur la classe DISQUE.

— **Amélioration de la classe mémoire** : La classe MÉMOIRE reste la plus difficile à identifier. Le rappel est passé à 79 % pour le CNN et le Transformer, réduisant ainsi la confusion avec la charge CPU.

— **Convergence des performances** : Le CNN 1D et le Transformer affichent des résultats similaires (94 % d'accuracy). Le modèle LSTM est toujours légèrement en dessous, même en augmentant le nombre d'époques à 25 (92 % d'accuracy).

En conclusion, la précision globale a augmenté sur tous les modèles. L'ajout de la variable sur la consommation électrique s'est révélé être un élément essentiel pour différencier le CPU de la MÉMOIRE.

Chapitre 7

Perspectives

Pour une utilisation réelle, capable d'identifier une application à partir d'une trace d'exécution (navigateur, compilateur, traitement de texte), plusieurs pistes sont possibles :

— **Fenêtres dynamiques** : Actuellement, les modèles utilisent des segments de données définis. En situation réelle, le système devra traiter un flux continu. Il faudrait alors rendre les fenêtres dynamiques ; cela permettrait d'analyser la signature en temps réel et de détecter les différentes étapes d'exécution.

— **Architecture** : Une approche possible serait d'utiliser une architecture de modèle à deux niveaux. Un premier modèle (comme réalisé dans ce projet) identifierait les séquences principales (CPU, Disque, Mémoire). Ses résultats serviraient d'entrées à un deuxième modèle, utilisé pour reconnaître l'enchaînement de ces phases afin de déterminer le type d'application.

Pour ce projet, nous avons utilisé l'IA Gemini pour la correction de bugs, la visualisation des résultats et l'aide à la rédaction du rapport.